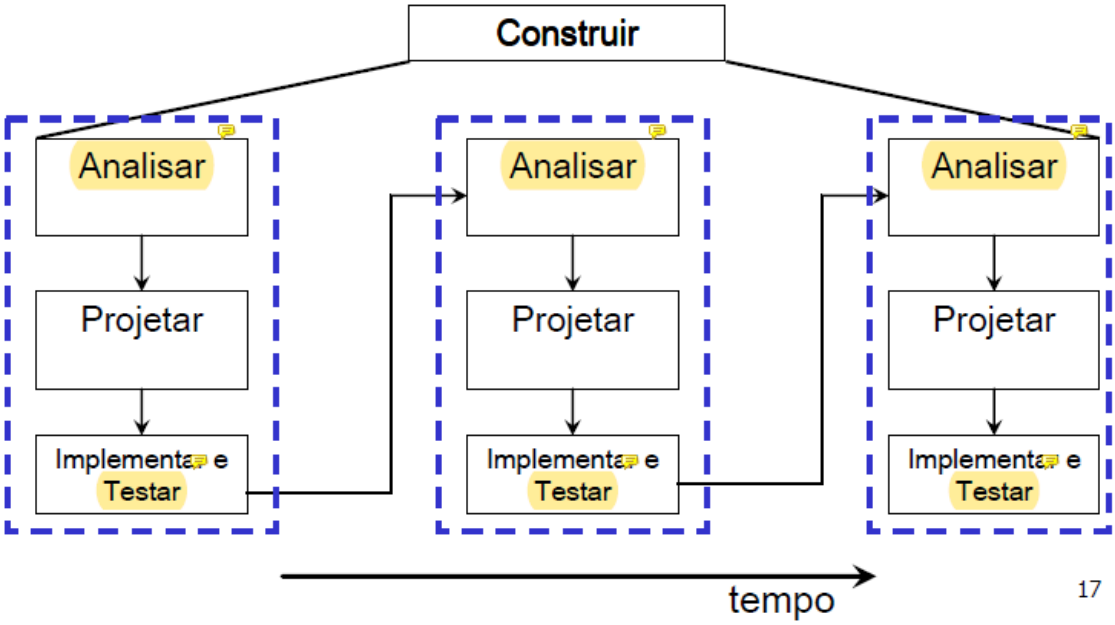
No Contexto do Interpretador MiniPar

Utilizando Componentes de Software e

Metodologias Ágeis

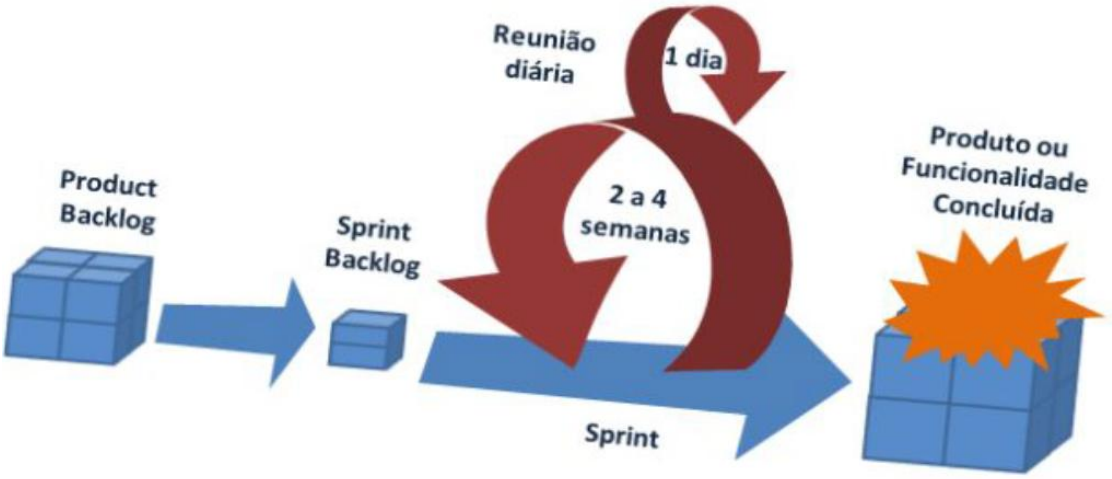
O SCRUM utiliza Desenvolvimento Incremental assim como outras Metodologias Ágeis (exemplo - Extreme Programming).

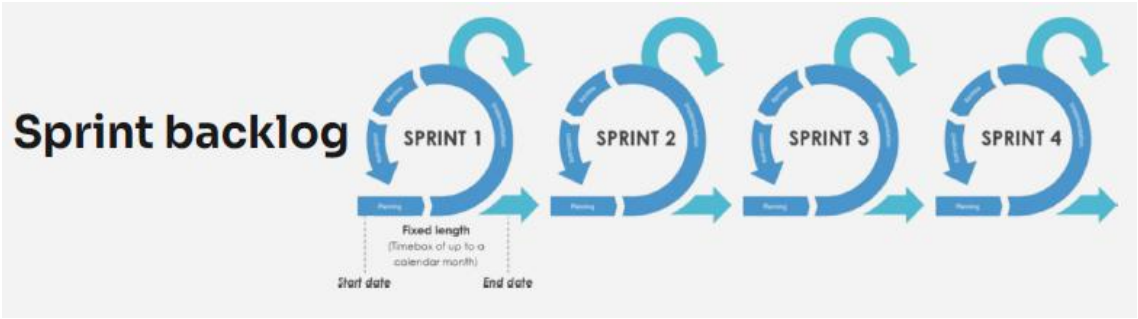
Exemplo de Desenvolvimento Incremental: o CDI (Ciclo de Desenvolvimento Iterativo e Incremental)



Agora no SCRUM, utiliza o conceito de Sprint que gera um incremento e para cada Sprint temos uma lista de funcionalidades a realizar (Sprint Backlog).

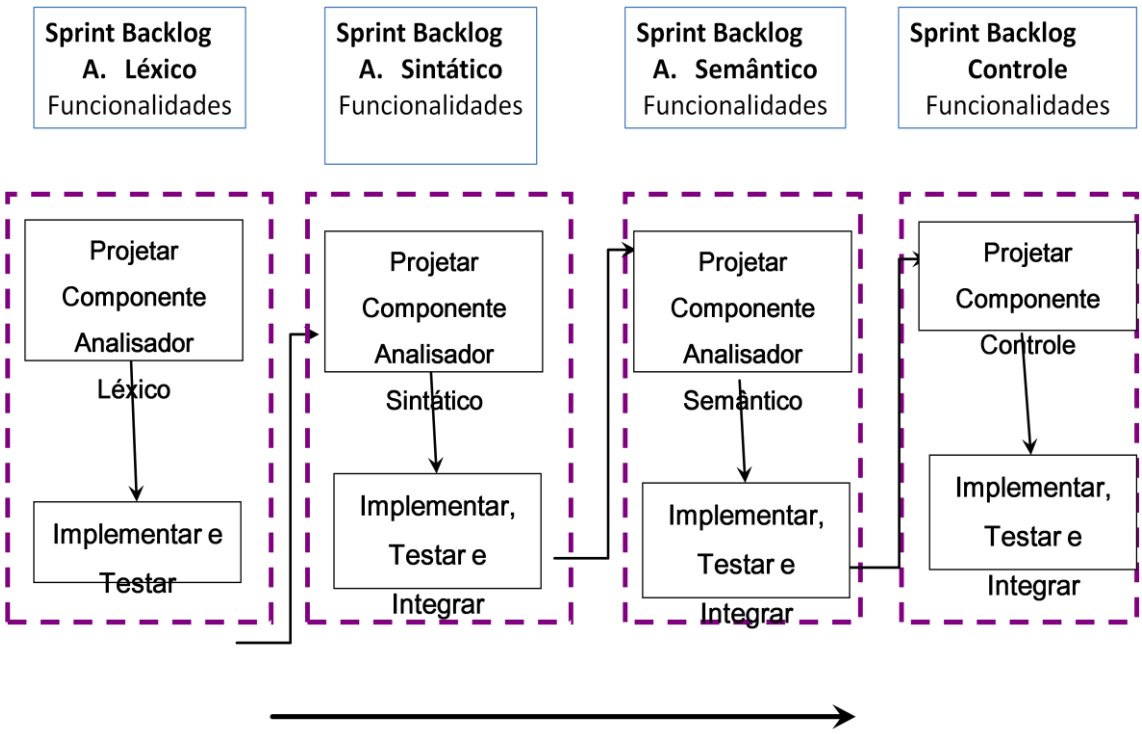
Sprint





O Scrum trabalha no Gerenciamento de Projetos.

Do ponto de vista de desenvolvimento de cada incremento do Scrum no contexto do Interpretador MiniPar, podemos ter o seguinte:



Observações:

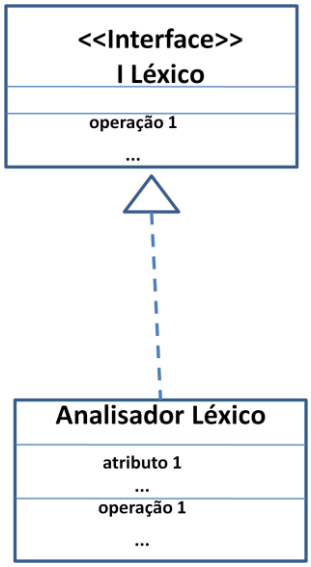
- O Product Backlog consiste na lista de funcionalidades do Sistema.

- O Sprint Backlog de cada Incremento corresponde a lista de funcionalidades de cada incremento, no caso do linterpretador MiniPar consiste um incremento ao desenvolvimento de um dos componentes do Interpretador MiniPar.

Exemplo: O Sprint Backlog do Componente Analisador Léxico seria:

- Desenvolver o Analisador Léxico junto com a Tabela de Símbolos.

Projetar um Componente do Interpretador MiniPar corresponde a fazer a modelagem via um diagrama de classes da Interface de cada componente e também da classe que realiza (implementa) cada interface, conforme mostrado no documento Projeto Final. Exemplo de componente em Java.



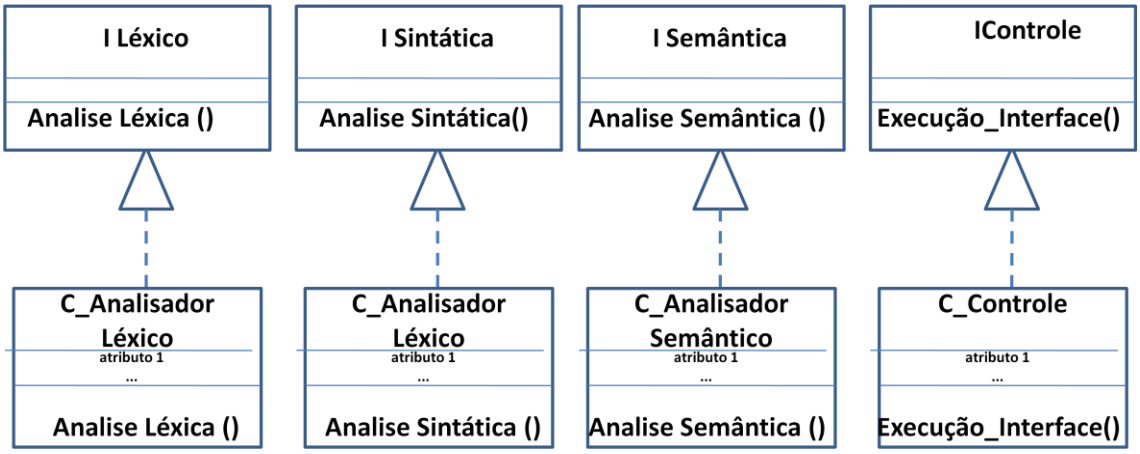
Os métodos da classe que implementam ou realizam a interface devem conter a implementação do algoritmo correspondente, seja Analisador Léxico, Analisador Sintático, ou Analisador Semântico.

Lembrete:

Para as linguagens de programação que não dão suporte a Interface como Java devem implementar a interface do componente como uma classe abstrata e a classe que realiza (implementa) a Interface deve ser representada como uma classe que herda da classe abstrata Interface e essa classe que herda deve fornecer a implementação dos métodos abstratos contidos na classe abstrata Interface.

Importante: (a ser realizado na implementação do interpretador MiniPar)

Também pode se implementar os componentes do Interpretador MiniPar com classes concretas **que qualquer linguagem de programação orientada a objetos fornece**. Exemplos: C++, Php, Phyton, e outros.



Temos dois tipos de classes concretas, a classe Interface começando pela letra **I** e a classe Componente começando pela letra **C**, cada classe componente fornece a implementação de cada Analisador Léxico, Sintático e Semântico, e também do Controle do Interpretador MiniPar.

Observação:

Uma classe concreta é aquela que a partir dela pode se criar instâncias ou objetos já que todos os métodos dessa classe contem sua implementação diferentemente das classes abstratas que não podem ser instanciadas devido a presença de pelo menos um método abstrato (que não tem implementação).

Como usar os componentes?

Resposta: via as Interfaces

Exemplo:

Declaramos um atributo de tipo Interface:

Ilexico ilexico; // declaração

...

// Criamos uma instância do componente C_Lexico

// e atribuímos a ilexico de tipo Interface

ilexico = new C_Lexico ();

// isto se interpreta como o atributo de interface léxico é

// realizado (implementado) pelo componente C_Lexico e

// esse componente fornece a implementação das

// operações de léxico

...

ilexico.analise_lexica ( );

// o método analise léxica será executado via o atributo de
// interface iléxico

Componentes com Classes Concretas

```
// Interface do Componente Analisador Lexico
public class Ilexico {
   // sem atributos
   public analise_lexica ( ) {
       // pode ser implementado com a instrução nula
       // ou com a implementação default do
       // Analisador Lexico
  }
}
// Classe Componente que implementa a interface
// Ilexico
public class C_Lexico extends Ilexico {
    // atributos
    public Token analise_lexica () {
       // implementação do algoritmo do Analisador
       // Lexico em Java
       // se houver implementação default
      // está implementação de analise_lexica
```

```
      // redefinirá a implementação default
    }
}
```

Da mesma forma para os outros componentes de software do Interpretador da linguagem MiniPar: Analisador Sintático, Analisador Semântico e o Controle.

```
// Interface do Componente Analisador Sintatico
public class ISintatica {
   // sem atributos
   public analise_sintatica ( ) {
       // pode ser implementado com a instrução nula
       // ou com a
       // implementação default do Analisador Sintático
  }
}
```

```
// Classe Componente que implementa a interface
// ISintatica
public class C_Sintatico extends ISintatica {
    // atributos
    public analise_sintatica () {
       // implementação do algoritmo do Analisador
       // Sintatico ou Parser em Java
       // se houver implementação default
      // está implementação de analise_ sintatica
      // redefinirá a implementação default
    }
}

// Interface do Componente Analisador Semântico
public class ISemantica {
   // sem atributos
   public analise_semantica ( ) {
       // pode ser implementado com a instrução nula
       // ou com a implementação default
       // do Analisador Semântico
```

```
  }
}

// Classe Componente que implementa a interface
// ISemantica
public class C_Semantico extends ISemantica {
    // atributos
    public analise_semantica () {
       // implementação do algoritmo do Analisador
       // Semântico em Java
       // se houver implementação default
      // está implementação de analise_ semantica
      // redefinirá a implementação default
    }
}

// Interface do Componente Controle
public class IControle {
   // sem atributos
   public execução_interface() {
       // pode ser implementado com a instrução nula
```

```
        // ou com a implementação default
        // da interface
    }
}

// Classe Componente que implementa a interface
// IControle
public class C_Controle extends IControle {
    // atributos
    public execução_interface(// parametros) {
        // implementação do algoritmo da Interface
        // em Java
        // se houver implementação default
       // está implementação da Interface
       // redefinirá a implementação default
    // o método execucao_interface() vai chamar na implementação dele
    // isintatico. analise_sintatica (ilexico, isemantico);
    // O componente Sintatico via o atributo de interface isintatico ou
    // parser vai chamar o método analise_sintatica que recbe como
    // parâmetros ilexico e isemantico,
    // o parser vai interagir com o analisador lexico (ilexico)
    // token a token, e o parser vai verificar a analise sintática e vai
```

```
    //  chamar o analisador semântico via isemantico para realizar a
    // verificação de tipos,
    // a interface também vai mostrar a saída ou resultado da
    // execução do programa MiniPar
    }
}
```

```
// A classe Interpretador da Linguagem MiniPar em Java
// versão a ser revisada, completada  e concluída pelos alunos

public class InterpretadorMIniPar {
// declaração das interfaces dos componentes
   public Ilexico ilexico;
   public ISintatico  isintatico;
   public ISemantico  isemantico;
   public IControle  icontrole;
   public InterpretadorMIniPar (// colocar parâmetros) {
        // instanciação das Interfaces via os Componentes
        ilexico = new C_Lexico ();
        isintatico = new C_Sintatico ();
        isemantico = new C_Semantico ();
        icontrole = new C_Controle ();
    }
   public void interface_interp (// parâmetros) {
       // apresentar a interface do interpretador
      // chamada ao controle via execução_interface ()
      icontrole.execucao_interface();
    }
}
```

```
// Executar o interpretador da linguagem Mini_Par
public class Execução_MiniPar {

    public static void main(String[] args) {
       InterpretadorMiniPar  intMiniPar;
       intMiniPar.interface_interp (// parâmetros);
   }

}
```